Huber Regressor

Nguyen Phu Nhan

Prerequisites: Linear Regression.

We define and process $\mathbf{X}, \mathbf{y}, \mathbf{w}$ similar to Linear Regression.

Huber Regressor extends Linear Regression by replacing the squared loss with the Huber loss, which is less sensitive to outliers. The resulting optimization problem is

$$\underset{\mathbf{w}}{\operatorname{argmin}} \left(\sum_{i=1}^{n} H_\epsilon \left(y_i - \mathbf{x}_i^\top \mathbf{w} \right) + \lambda \|\mathbf{w}\|_2^2 \right),$$

where

$$H_\epsilon(r) = \begin{cases} r^2, & |r| \le \epsilon, \\ 2\epsilon|r| - \epsilon^2, & |r| > \epsilon. \end{cases}$$

Here, $r = y_i - \mathbf{x}_i^\top \mathbf{w}$ is the residual, ϵ controls the point at which the loss changes from quadratic to linear, and $\lambda \ge 0$ controls the strength of L_2 regularization.

Unlike ordinary Linear Regression or Ridge, Huber Regressor does not have a closed-form solution for $\mathbf{w}$ because the Huber loss is piecewise-defined. Therefore, $\mathbf{w}$ is usually obtained using iterative optimization methods, such as gradient descent.

Let

$$r_i = y_i - \mathbf{x}_i^\top \mathbf{w}.$$

The gradient-based update rule is

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + \eta \sum_{i=1}^{n} \psi_\epsilon(r_i) \mathbf{x}_i - 2\eta\lambda \mathbf{w}^{(t)},$$

where

$$\psi_\epsilon(r_i) = \begin{cases} 2r_i, & |r_i| \le \epsilon, \\ 2\epsilon \operatorname{sign}(r_i), & |r_i| > \epsilon. \end{cases}$$

Here, η is the learning rate, ϵ controls the threshold between squared loss and absolute loss, and λ controls the strength of L_2 regularization.